SMART CAMPUS

1. Contexte et mission

Dans le cadre du développement d'un *Smart Campus*, on souhaite déployer une plateforme cloud pour superviser des équipements et des services connectés sur son campus : consommation énergétique des bâtiments et éclairage extérieur.

Des capteurs et des équipements IoT produisent des données qui permettent de suivre le fonctionnement de ces services et de repérer des situations anormales. Votre équipe intervient comme **équipe d'ingénierie cloud et réseaux** : elle conçoit et déploie une plateforme qui reçoit les données, les traite, les stocke et les affiche dans une interface de supervision, avec des alertes adaptées au cas d'usage choisi.

Le projet met l'accent sur **les architectures cloud et IoT, la conteneurisation, les communications réseau, l'automatisation et la sécurisation des services.** Les équipements physiques sont **simulés** : un jeu de données est rejoué sur la plateforme comme le feraient de vrais capteurs.

2. Objectifs

À l'issue du projet, vous serez capables de :

- concevoir l'architecture d'une plateforme cloud pour une application IoT et justifier vos choix ;
- déployer plusieurs services communicants dans un environnement conteneurisé, de façon reproductible ;
- collecter, décoder, valider et traiter des données issues d'équipements connectés simulés ;
- stocker et visualiser des mesures ou des événements horodatés ;
- détecter des situations anormales et générer des alertes pertinentes ;
- automatiser des opérations de déploiement et appliquer des principes de sécurité réseau (authentification, secrets, exposition minimale) ;
- tester, documenter et présenter une application fonctionnelle.

3. Les cas d'usage

Chaque binôme choisit **un cas d'usage** parmi les deux suivants. Les deux cas utilisent la même architecture ; les données, les indicateurs et les règles d'alerte diffèrent.

Cas A : Supervision énergétique d'un bâtiment

Suivre la consommation électrique de compteurs répartis dans plusieurs bâtiments, visualiser son évolution et signaler un dépassement de seuil ou une absence de remontée de mesures.

- **Équipements simulés :** trois compteurs, deux bâtiments, une mesure de puissance (en watts) toutes les 5 minutes.
- **Situations à repérer :** un dépassement de consommation, un compteur qui ne répond plus pendant une période prolongée, une valeur impossible.

Cas B : Éclairage extérieur intelligent

Suivre l'état des luminaires (allumé ou éteint), leur consommation et la luminosité mesurée, et signaler un fonctionnement anormal.

- **Équipements simulés :** quatre luminaires, deux zones, une mesure d'état, de puissance et de luminosité toutes les 5 minutes.
- **Plage de fonctionnement prévue :** allumés de 18 h à 6 h (heure UTC), éteints le reste du temps.
- **Situations à repérer :** un luminaire allumé hors de la plage prévue, une panne présumée (état allumé mais consommation quasi nulle), un luminaire qui ne répond plus, une valeur impossible.

4. Données et matériel fournis

Les ressources du projet sont :

- un générateur de jeu de données (generer_donnees.py), à utiliser avec une graine propre à chaque équipe : les données et les anomalies varient selon la graine ;
- un **exemple de script de rejeu** (rejeu.py) qui publie les mesures sur un broker MQTT ;
- la **spécification des données** (champs, sujets MQTT, format des messages, nature des anomalies) ;

Le rejeu simule l'envoi de messages IoT sans capteur physique. Il **ne constitue pas** une transmission radio LoRaWAN réelle.

5. Architecture attendue

La plateforme devra assurer les fonctions suivantes :

1. **Acquisition** : réception des messages issus du rejeu via un protocole IoT, notamment MQTT ;
2. **Traitement** : décodage, vérification et préparation des données ;
3. **Stockage** : enregistrement des mesures ou événements dans une base de données adaptée ;
4. **Visualisation** : présentation des indicateurs dans un tableau de bord ;
5. **Surveillance** : détection d'anomalies et génération d'alertes ;
6. **Déploiement** : mise en place reproductible des services dans un environnement conteneurisé.

Jeu de données → Rejeu Python → Broker MQTT → Traitement → Base de données → Tableau de bord / alertes
Déploiement reproductible : Docker Compose (ou solution équivalente)

Une architecture reposant sur **Docker Compose, Mosquitto, Node-RED, InfluxDB et Grafana** constitue une solution possible. Des technologies équivalentes peuvent être proposées, **à condition de justifier les choix** et d'obtenir l'accord de l'enseignant avant de vous engager.

Contraintes techniques :

- tous les services de la plateforme sont conteneurisés ;
- les versions des images et des outils sont fixées (pas de dépendance à une version « la plus récente » non maîtrisée) ;
- la plateforme démarre à partir d'une **procédure documentée**, sur une machine vierge disposant de Docker ;
- les mesures sont enregistrées avec leur **horodatage d'origine** (celui de la mesure, pas celui de la réception) ;
- la configuration (tableau de bord, flux de traitement, source de données) est **versionnée** : elle ne se reconstruit pas à la main.

6. Étapes de réalisation

Pour chaque étape, le tableau précise le travail attendu, ce que vous devez pouvoir **montrer**, et le critère qui permet de le considérer comme atteint.

Étape	Travail attendu	Ce que vous devez pouvoir montrer
E1 — Conception	Choisir le cas d'usage, proposer une architecture, identifier les services, leurs rôles et leurs communications	Schéma d'architecture, tableau des services (rôle, technologie, ports), plan des réseaux, justification des choix
E2 — Déploiement	Déployer une première plateforme conteneurisée et vérifier les communications entre services	Démarrage en une procédure ; preuve que chaque service joint les autres (journaux, commandes)
E3 — Intégration IoT	Rejouer les données, les traiter et les enregistrer dans une base	Messages reçus, décodés, validés et visibles dans la base
E4 — Visualisation et alertes	Construire un tableau de bord et détecter au moins deux situations anormales adaptées au cas d'usage	Tableau de bord de trois indicateurs au moins ; alertes visibles ou enregistrées
E5 — Automatisation et sécurité	Automatiser le démarrage des services, configurer une authentification et protéger	Une commande démarre tout ; accès anonyme au broker refusé ; aucun secret

	les informations sensibles	dans le dépôt
E6 — Validation	Tester le fonctionnement global et réaliser un approfondissement technique au choix	Recette de bout en bout depuis une machine propre ; approfondissement démontré
E7 — Soutenance	Présenter l'architecture, justifier les choix et démontrer le fonctionnement	Présentation, démonstration, réponses aux questions

7. Fonctionnalités minimales

Pour être considéré comme fonctionnel, le projet devra permettre de :

- démarrer les services à l'aide d'une procédure documentée ;
- rejouer les données d'au moins deux capteurs ou équipements simulés ;
- transmettre les données via un broker MQTT ;
- enregistrer les données avec leurs horodatages et identifiants ;
- afficher au moins trois indicateurs ou graphiques pertinents pour le cas d'usage ;
- détecter **deux situations anormales**, dont **une absence prolongée de données** et une anomalie propre au cas choisi (dépassement de consommation, état d'éclairage anormal, par exemple) ;
- générer une alerte visible ou enregistrée ;
- protéger l'accès au broker par authentification et **ne pas placer de mots de passe dans le dépôt Git**.

Points d'attention :

- Les seuils d'alerte se déduisent de l'**observation de vos données**, pas de valeurs recopiées : vous expliquerez votre démarche.
- Le rejeu **accélère le temps** : une détection d'absence de données fondée sur l'horloge de votre machine ne se comporte pas comme sur des mesures en temps réel. Choisissez et justifiez l'horloge de référence.
- Une alerte utile se déclenche une fois par situation, pas à chaque mesure anormale.

8. Approfondissement technique

Chaque binôme choisit **un approfondissement** parmi les cinq suivants. Il doit être **justifié, documenté** et accompagné d'une **démonstration** ou d'éléments de validation.

Option	Objectif
A — Automatisation de l'infrastructure	Préparer un environnement et déployer les services avec un outil adapté (Ansible, OpenTofu ou équivalent)
B — Intégration et déploiement continus	Mettre en place un pipeline CI/CD
C — Orchestration	Déployer une partie de la plateforme sur un orchestrateur léger (k3s, kind ou équivalent)
D — Sécurité réseau	Renforcer la sécurité par l'isolation des services, les contrôles d'accès et la sécurisation TLS des communications MQTT
E — Supervision avancée	Collecter des métriques d'infrastructure et générer une alerte sur l'état d'un service ou l'utilisation des ressources

9. Livrables

Le dépôt Git de l'équipe contient :

1. **le code et la configuration** nécessaires pour démarrer la plateforme (fichiers de déploiement, configuration des services, flux de traitement, tableaux de bord) ;

2. **un README** suivant le modèle fourni : description, architecture, démarrage, détection d'anomalies, sécurité, approfondissement, tests, choix et limites, répartition du travail, sources ;
3. **un schéma d'architecture** ;
4. **la procédure de rejeu** et d'obtention du jeu de données ;
5. **un fichier de déclaration des contributions** (qui a fait quoi)

Mode de dépôt : dépôt privé partagé avec l'enseignant (compte personnel sur une forge de votre choix) **ou** archive du dépôt, dossier `.git` compris, déposée sur la plateforme pédagogique. Le mode exact vous est précisé en séance 1.

Ne figurent jamais dans le dépôt : mots de passe, jetons, clés privées, fichiers `.env`. Un secret retrouvé dans l'historique est sanctionné, même s'il a été supprimé ensuite.

10. Soutenance (le 16/12)

Chaque équipe dispose de **10 minutes** de présentation et de démonstration, suivies de **5 minutes de questions**, posées à **chaque membre** du binôme.

Contenu attendu :

1. le cas d'usage et l'architecture (schéma) ;
2. la démonstration : démarrage, rejeu, tableau de bord, alertes, et l'approfondissement ;
3. les choix techniques, les difficultés rencontrées, les limites et les pistes d'amélioration.

Conseils : répétez la démonstration de bout en bout, depuis zéro ; prévoyez un plan B (captures ou enregistrement) en cas de panne de réseau ; chacun doit pouvoir expliquer **toutes** les parties, y compris celles qu'il n'a pas écrites.